EXPERIMENT – 7

AIM: To identify functional dependencies in a denormalized Sales table and normalize the relation step-by-step from Unnormalized Form (UNF) to First Normal Form (1NF), Second Normal Form (2NF), Third Normal Form (3NF), and Boyce–Codd Normal Form (BCNF). To verify lossless join and dependency preservation using SQL queries.

SOURCE CODE:

1. CREATE DATABASE

```
CREATE DATABASE RetailNormalization;
USE RetailNormalization;
```

2. CREATE THE DENORMALIZED SALES TABLE (UNF)

```
CREATE TABLE Purchase_UNF (
    Order_ID INT,
    Order_Date DATE,
    Customer_ID VARCHAR(10),
    Customer_Name VARCHAR(50),
    Product_ID VARCHAR(10),
    Product_Name VARCHAR(50),
    Quantity INT,
    Unit_Price DECIMAL(10,2),
    Salesperson_ID VARCHAR(10),
    Salesperson_Name VARCHAR(50)
);

INSERT INTO Purchase_UNF VALUES
(2001, '2026-02-05', 'C11', 'Kabir', 'P11', 'Tablet', 2,
32000, 'S11', 'Ananya'),

(2001, '2026-02-05', 'C11', 'Kabir', 'P12', 'Headphones', 3,
1800, 'S11', 'Ananya'),

(2002, '2026-02-08', 'C12', 'Ishita', 'P11', 'Tablet', 1,
32000, 'S12', 'Vikram'),

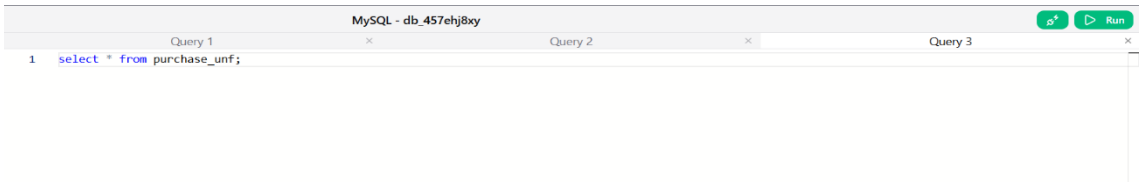
(2003, '2026-02-12', 'C13', 'Dev', 'P13', 'Printer', 2,
12500, 'S11', 'Ananya'),

(2004, '2026-02-16', 'C11', 'Kabir', 'P13', 'Printer', 1,
12500, 'S12', 'Vikram'),

(2005, '2026-02-20', 'C14', 'Naina', 'P12', 'Headphones', 4,
1800, 'S13', 'Arjun');

SELECT * FROM Purchase_UNF;
```

OUTPUT:



Order_ID	Order_Date	Customer_ID	Customer_Name	Product_ID	Product_Name	Quantity	Unit_Price	Salesperson_ID	Salesperson_Name
2001	2026-02-05T00:00:00.000Z	C11	Kabir	P11	Tablet	2	32000.00	S11	Ananya
2001	2026-02-05T00:00:00.000Z	C11	Kabir	P12	Headphones	3	1800.00	S11	Ananya
2002	2026-02-08T00:00:00.000Z	C12	Ishita	P11	Tablet	1	32000.00	S12	Vikram
2003	2026-02-12T00:00:00.000Z	C13	Dev	P13	Printer	2	12500.00	S11	Ananya
2004	2026-02-16T00:00:00.000Z	C11	Kabir	P13	Printer	1	12500.00	S12	Vikram
2005	2026-02-20T00:00:00.000Z	C14	Naina	P12	Headphones	4	1800.00	S13	Arjun

3. IDENTIFY FUNCTIONAL DEPENDENCIES

No.	Functional Dependency	Meaning
1	`Order_ID → Order_Date`	An order determines its date.
2	`Order_ID → Customer_ID`	An order determines its customer.
3	`Order_ID → Salesperson_ID`	An order determines its salesperson.
4	`Customer_ID → Customer_Name`	A customer ID determines the customer name.
5	`Product_ID → Product_Name`	A product ID determines its name.
6	`Product_ID → Unit_Price`	A product ID determines its price.
7	`Salesperson_ID → Salesperson_Name`	A salesperson ID determines the salesperson name.
8	`(Order_ID, Product_ID) → Quantity`	An order-product pair determines the quantity.

Candidate Key: `(Order_ID, Product_ID)`

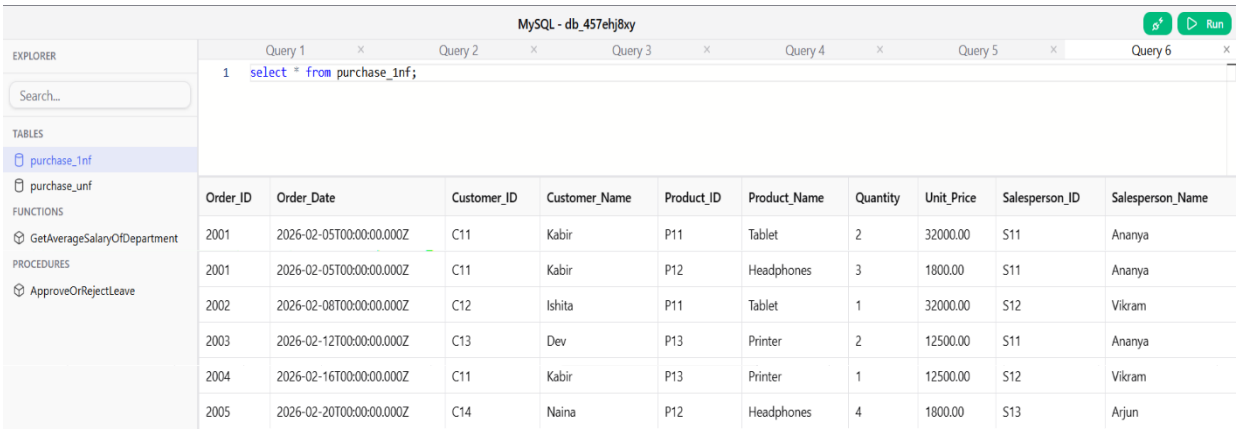
SQL QUERIES:

4. FIRST NORMAL FORM (1NF)

```
CREATE TABLE Purchase_1NF (
    Order_ID INT,
    Order_Date DATE,
    Customer_ID VARCHAR(10),
    Customer_Name VARCHAR(50),
    Product_ID VARCHAR(10),
    Product_Name VARCHAR(50),
    Quantity INT,
    Unit_Price DECIMAL(10,2),
    Salesperson_ID VARCHAR(10),
    Salesperson_Name VARCHAR(50),
    PRIMARY KEY (Order_ID, Product_ID)
);
```

```
INSERT INTO Purchase_1NF
SELECT * FROM Purchase_UNF;
SELECT * FROM Purchase_1NF;
```

OUTPUT:



Order_ID	Order_Date	Customer_ID	Customer_Name	Product_ID	Product_Name	Quantity	Unit_Price	Salesperson_ID	Salesperson_Name
2001	2026-02-05T00:00:00.000Z	C11	Kabir	P11	Tablet	2	32000.00	S11	Ananya
2001	2026-02-05T00:00:00.000Z	C11	Kabir	P12	Headphones	3	1800.00	S11	Ananya
2002	2026-02-08T00:00:00.000Z	C12	Ishita	P11	Tablet	1	32000.00	S12	Vikram
2003	2026-02-12T00:00:00.000Z	C13	Dev	P13	Printer	2	12500.00	S11	Ananya
2004	2026-02-16T00:00:00.000Z	C11	Kabir	P13	Printer	1	12500.00	S12	Vikram
2005	2026-02-20T00:00:00.000Z	C14	Naina	P12	Headphones	4	1800.00	S13	Arjun

5. SECOND NORMAL FORM (2NF)

The candidate key is: `(Order_ID, Product_ID)`

PARTIAL DEPENDENCIES

```
Order_ID → Order_Date, Customer_ID, Customer_Name,
           Salesperson_ID, Salesperson_Name

Product_ID → Product_Name, Unit_Price
```

DECOMPOSITION INTO 2NF

Table 1: Orders

```
CREATE TABLE Orders (
    Order_ID INT PRIMARY KEY,
    Order_Date DATE,
    Customer_ID VARCHAR(10),
    Customer_Name VARCHAR(50),
    Salesperson_ID VARCHAR(10),
    Salesperson_Name  VARCHAR(50)
);

INSERT INTO Orders
SELECT DISTINCT
    Order_ID,
    Order_Date,
    Customer_ID,
    Customer_Name,
```

```
    Salesperson_ID,
    Salesperson_Name
FROM Purchase_1NF;
```

Table 2: Products

```
CREATE TABLE Products (
    Product_ID VARCHAR(10) PRIMARY KEY,
    Product_Name VARCHAR(50),
    Unit_Price DECIMAL(10,2)
);

INSERT INTO Products
SELECT DISTINCT
    Product_ID,
    Product_Name,
    Unit_Price
FROM Purchase_1NF;
```

Table 3: Order_Details

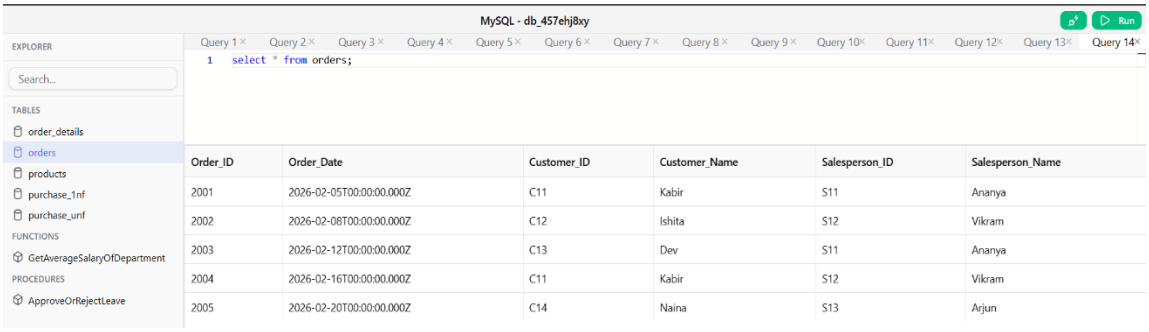
```
CREATE TABLE Order_Details (
    Order_ID INT,
    Product_ID VARCHAR(10),
    Quantity INT,
    PRIMARY KEY (Order_ID, Product_ID),
    FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID),
    FOREIGN KEY (Product_ID) REFERENCES Products(Product_ID)
);

INSERT INTO Order_Details
SELECT Order_ID, Product_ID, Quantity
FROM Purchase_1NF;
```

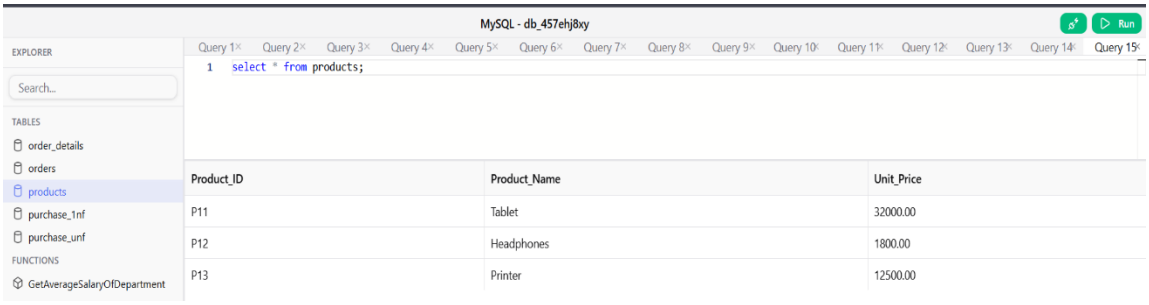
Verify

```
SELECT * FROM Orders;
SELECT * FROM Products;
SELECT * FROM Order_Details;
```

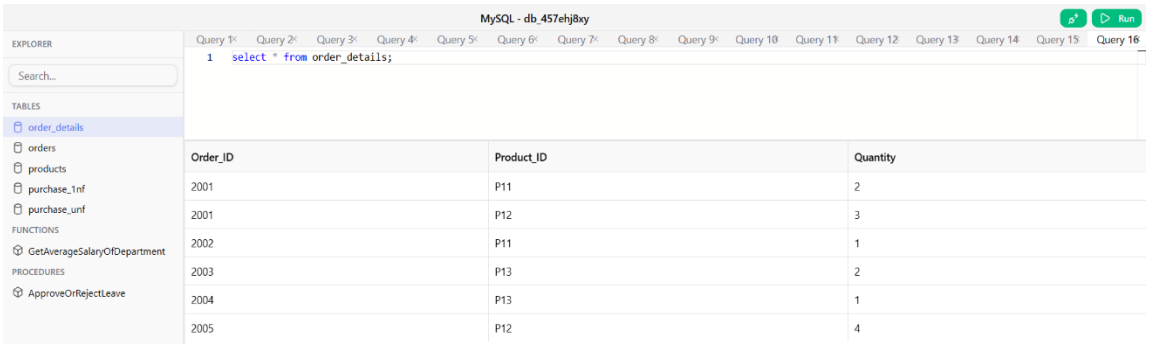
OUTPUT:



Order_ID	Order_Date	Customer_ID	Customer_Name	Salesperson_ID	Salesperson_Name
2001	2026-02-05T00:00:00.000Z	C11	Kabir	S11	Ananya
2002	2026-02-08T00:00:00.000Z	C12	Ishita	S12	Vikram
2003	2026-02-12T00:00:00.000Z	C13	Dev	S11	Ananya
2004	2026-02-16T00:00:00.000Z	C11	Kabir	S12	Vikram
2005	2026-02-20T00:00:00.000Z	C14	Naina	S13	Arjun



Product_ID	Product_Name	Unit_Price
P11	Tablet	32000.00
P12	Headphones	1800.00
P13	Printer	12500.00



Order_ID	Product_ID	Quantity
2001	P11	2
2001	P12	3
2002	P11	1
2003	P13	2
2004	P13	1
2005	P12	4

6. THIRD NORMAL FORM (3NF)

IDENTIFY TRANSITIVE DEPENDENCIES

Order_ID → Customer_ID → Customer_Name

Order_ID → Salesperson_ID → Salesperson_Name

Therefore:

Customer_ID → Customer_Name

Salesperson_ID → Salesperson_Name

Table 1: Customers

```
CREATE TABLE Customers (
    Customer_ID VARCHAR(10) PRIMARY KEY,
    Customer_Name VARCHAR(50)
);

INSERT INTO Customers
SELECT DISTINCT Customer_ID, Customer_Name
FROM Purchase_1NF;
```

Table 2: Salespersons

```
CREATE TABLE Salespersons (
    Salesperson_ID VARCHAR(10) PRIMARY KEY,
    Salesperson_Name VARCHAR(50)
);

INSERT INTO Salespersons
SELECT DISTINCT Salesperson_ID, Salesperson_Name
FROM Purchase_1NF;
```

Table 3: Orders_3NF

```
CREATE TABLE Orders_3NF (
    Order_ID INT PRIMARY KEY,
    Order_Date DATE,
    Customer_ID VARCHAR(10),
    Salesperson_ID VARCHAR(10),

    FOREIGN KEY (Customer_ID)
        REFERENCES Customers(Customer_ID),

    FOREIGN KEY (Salesperson_ID)
        REFERENCES Salespersons(Salesperson_ID)
);

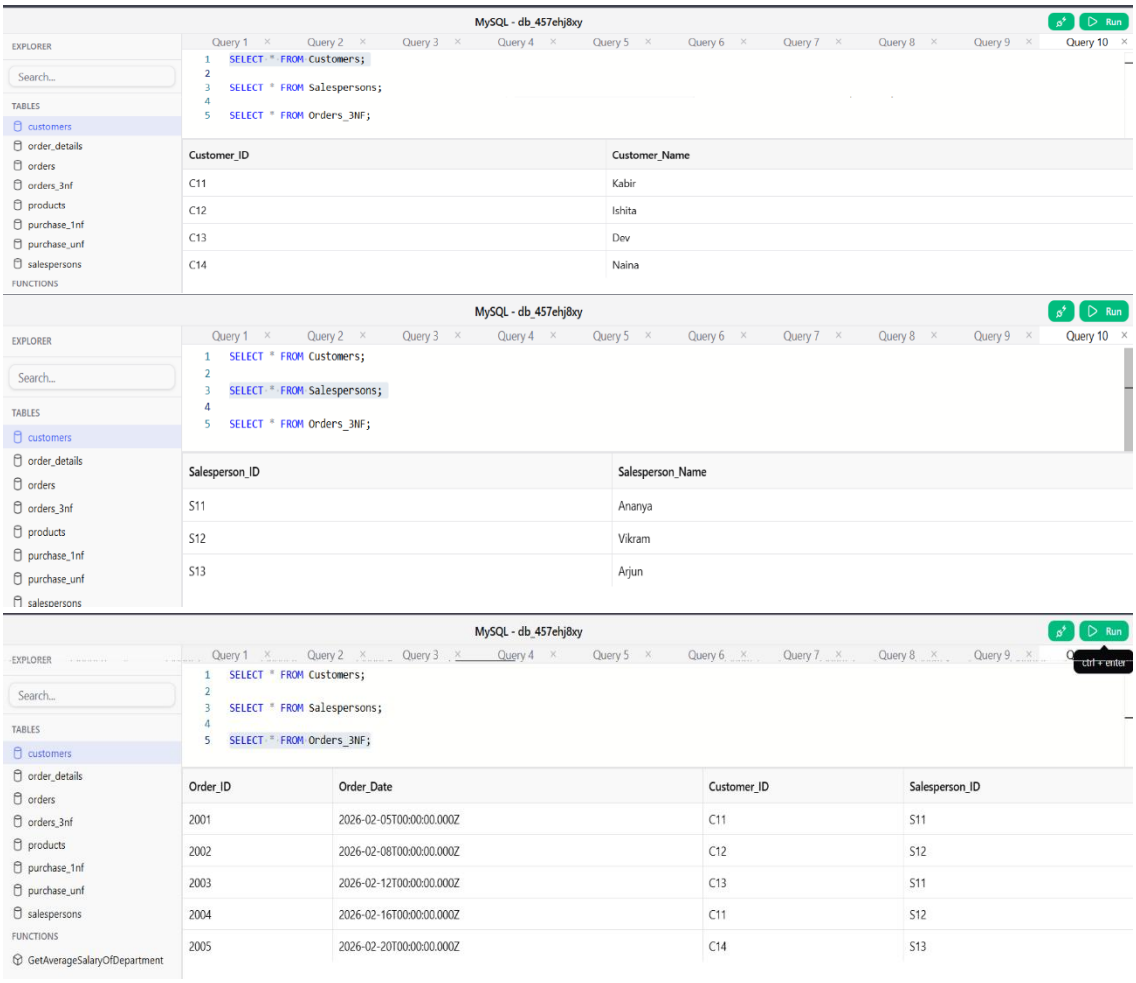
INSERT INTO Orders_3NF
SELECT DISTINCT
    Order_ID,
    Order_Date,
    Customer_ID,
    Salesperson_ID
FROM Purchase_1NF;
```

Verify

```
SELECT * FROM Customers;
SELECT * FROM Salespersons;
```

```
SELECT * FROM Orders_3NF;
```

OUTPUT:



7. BOYCE–CODD NORMAL FORM (BCNF)

```
Employee_Assignment(
     Employee_ID,
     Project_ID,
     Supervisor_ID
)
```

Functional dependencies:

```
(Employee_ID, Project_ID) → Supervisor_ID

Supervisor_ID → Project_ID
```

Create Table

```
CREATE TABLE Consultant_Assignment (
    Consultant_ID VARCHAR(10),
    Project_ID VARCHAR(10),
    Coordinator_ID VARCHAR(10),
    PRIMARY KEY (Consultant_ID, Project_ID)
);

INSERT INTO Consultant_Assignment VALUES
('E11', 'P11', 'S11'),
('E12', 'P11', 'S11'),
('E13', 'P12', 'S12'),
('E14', 'P12', 'S12');
```

Here, `Coordinator_ID → Project_ID` violates BCNF because `Coordinator_ID` is not a superkey. However, the relation satisfies 3NF because `Project_ID` is a prime attribute under the stated dependencies.

DECOMPOSE INTO BCNF

Table 1: Supervisor_Project

```
CREATE TABLE Coordinator_Project (
    Coordinator_ID VARCHAR(10) PRIMARY KEY,
    Project_ID VARCHAR(10)
);

INSERT INTO Coordinator_Project VALUES
('S11', 'P11'),
('S12', 'P12');
```

Table 2: Employee_Supervisor

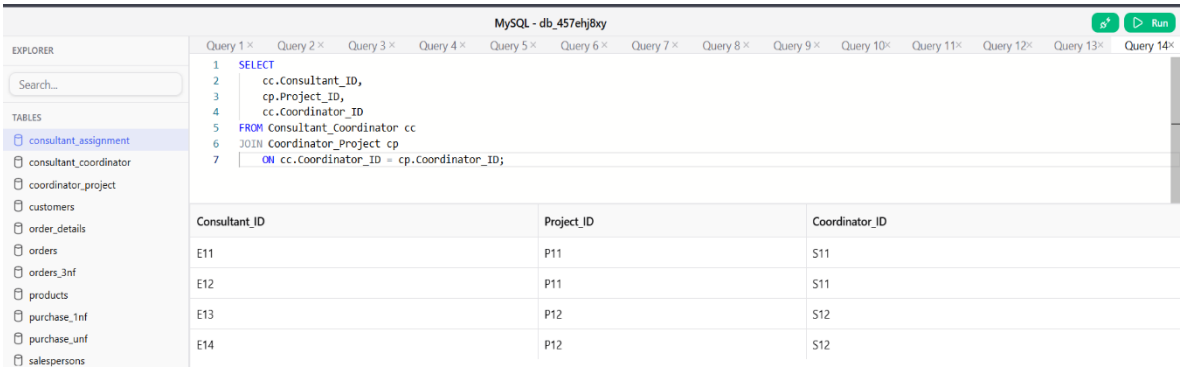
```
CREATE TABLE Consultant_Coordinator (
    Consultant_ID VARCHAR(10),
    Coordinator_ID VARCHAR(10),
    PRIMARY KEY (Consultant_ID, Coordinator_ID)
);

INSERT INTO Consultant_Coordinator VALUES
('E11', 'S11'),
('E12', 'S11'),
('E13', 'S12'),
('E14', 'S12');
```

Verify BCNF Decomposition

```
SELECT
    cc.Consultant_ID,
    cp.Project_ID,
    cc.Coordinator_ID
FROM Consultant_Coordinator cc
JOIN Coordinator_Project cp
    ON cc.Coordinator_ID = cp.Coordinator_ID;
```

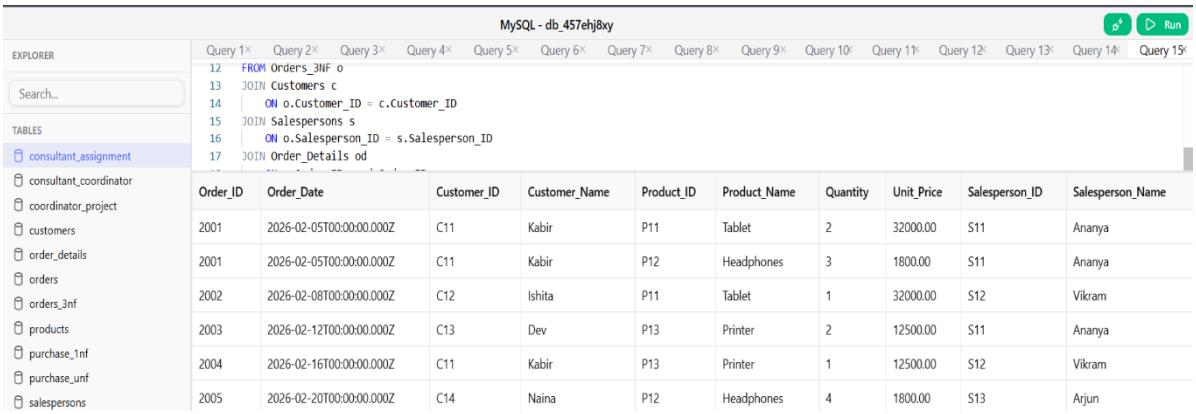
OUTPUT:



8. VERIFY LOSSLESS JOIN

```
    SELECT
        o.Order_ID,
        o.Order_Date,
        c.Customer_ID,
        c.Customer_Name,
        od.Product_ID,
        p.Product_Name,
        od.Quantity,
        p.Unit_Price,
        s.Salesperson_ID,
        s.Salesperson_Name
    FROM Orders_3NF o
    JOIN Customers c
        ON o.Customer_ID =
c.Customer_ID
    JOIN Salespersons s
        ON o.Salesperson_ID =
s.Salesperson_ID
    JOIN Order_Details od
        ON o.Order_ID = od.Order_ID
    JOIN Products p
        ON od.Product_ID =
p.Product_ID
    ORDER BY o.Order_ID,
od.Product_ID;
```

OUTPUT:

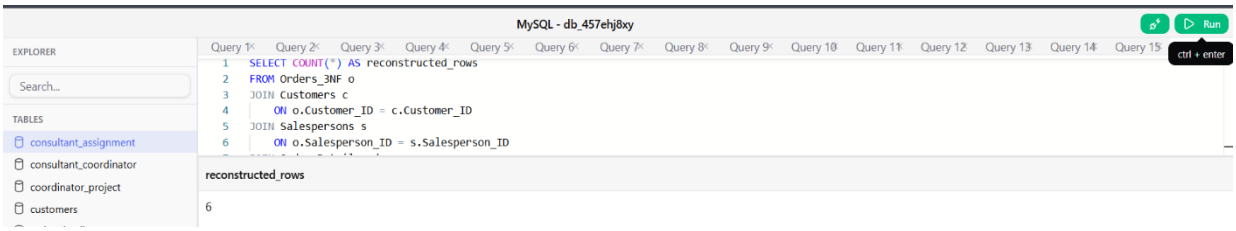


Order_ID	Order_Date	Customer_ID	Customer_Name	Product_ID	Product_Name	Quantity	Unit_Price	Salesperson_ID	Salesperson_Name
2001	2026-02-05T00:00:00.000Z	C11	Kabir	P11	Tablet	2	32000.00	S11	Ananya
2001	2026-02-05T00:00:00.000Z	C11	Kabir	P12	Headphones	3	1800.00	S11	Ananya
2002	2026-02-08T00:00:00.000Z	C12	Ishita	P11	Tablet	1	32000.00	S12	Vikram
2003	2026-02-12T00:00:00.000Z	C13	Dev	P13	Printer	2	12500.00	S11	Ananya
2004	2026-02-16T00:00:00.000Z	C11	Kabir	P13	Printer	1	12500.00	S12	Vikram
2005	2026-02-20T00:00:00.000Z	C14	Naina	P12	Headphones	4	1800.00	S13	Arjun

Verify Row Count

```
SELECT COUNT(*) AS reconstructed_rows
FROM Orders_3NF o
JOIN Customers c
    ON o.Customer_ID = c.Customer_ID
JOIN Salespersons s
    ON o.Salesperson_ID = s.Salesperson_ID
JOIN Order_Details od
    ON o.Order_ID = od.Order_ID
JOIN Products p
    ON od.Product_ID = p.Product_ID;
```

OUTPUT:



G. VERIFY DEPENDENCY PRESERVATION

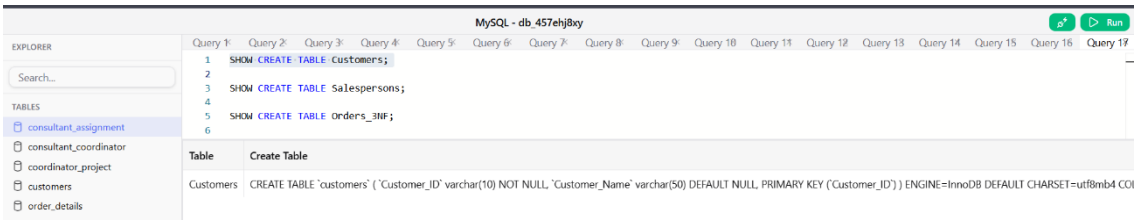
Functional Dependencies and Their Relations

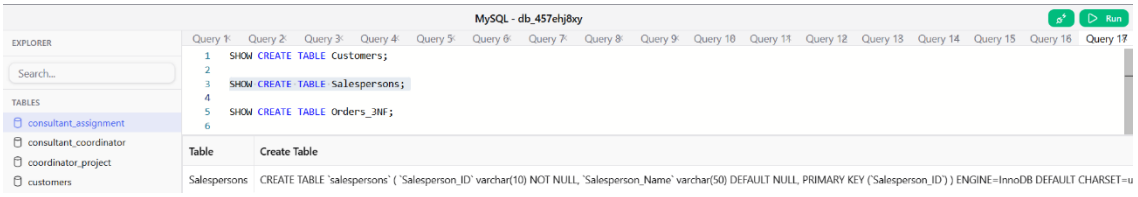
Functional Dependency	Relation
Customer_ID → Customer_Name	Customers
Salesperson_ID → Salesperson_Name	Salespersons
Order_ID → Order_Date, Customer_ID, Salesperson_ID	Orders_3NF
Product_ID → Product_Name, Unit_Price	Products
(Order_ID, Product_ID) → Quantity	Order_Details

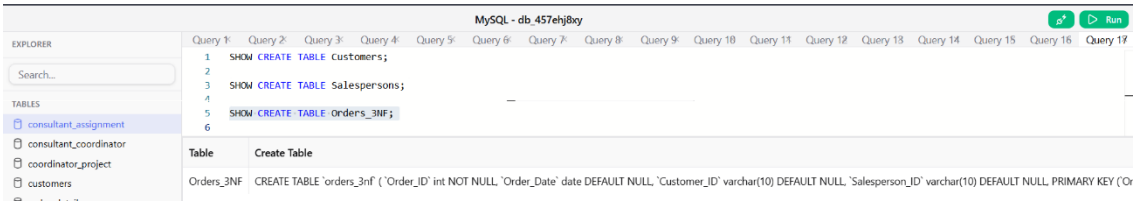
SQL Verification

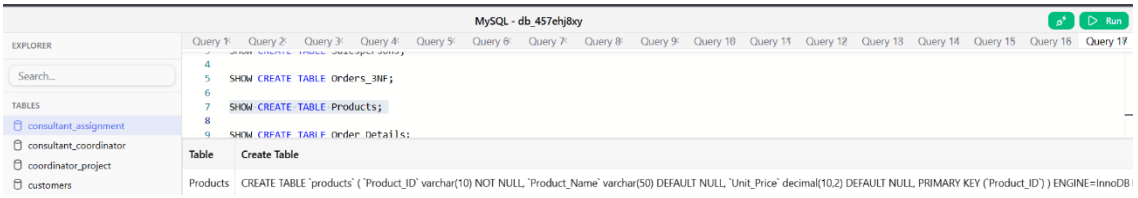
```
SHOW CREATE TABLE Customers;
SHOW CREATE TABLE Salespersons;
SHOW CREATE TABLE Orders_3NF;
SHOW CREATE TABLE Products;
SHOW CREATE TABLE Order_Details;
```

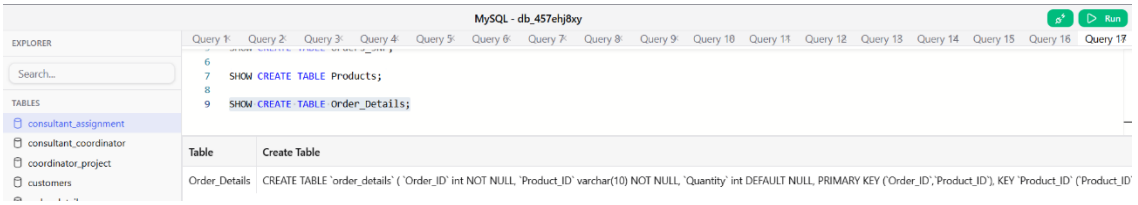
OUTPUT:











10. COMPARISON OF NORMAL FORMS

Normal Form	Main Requirement
UNF	May contain repeating groups or non-atomic values
1NF	Atomic values and no repeating groups
2NF	1NF and no partial dependency of non-prime attributes on a candidate key
3NF	2NF and no prohibited transitive dependencies
BCNF	Every determinant of a non-trivial functional dependency is a superkey

RESULT: The denormalized Sales table was analyzed for functional dependencies and normalized step-by-step from UNF to 1NF, 2NF, and 3NF. A separate relation was decomposed into BCNF to demonstrate the stricter BCNF requirement. Lossless reconstruction and dependency preservation were examined using SQL queries.